Interview questions and answers

What an interviewer may ask about this project, with suggested answers

Answers are written in the first person so you can adapt them. Be ready to open the code at the file named in each topic.

A. Design and approach

Q. Walk me through what you built in one minute.

I added two features to a CRM: moving a payment recorded against the wrong customer (FDSK-169) and correcting an overstated delivery (FDSK-170). Before doing so, I mapped the existing code against the canonical business rules and found the code already had a good action pipeline, but role checks and cash recalculation were repeated by hand, one action could only touch one customer, and the return-settlement logic was stuck inside Returns. I fixed those three things with a small rules table, a shell that reconciles every touched customer, and a shared release primitive. Then each feature became a small, local change.

Q. Why didn't you use a rules engine like Drools?

The rules here are arithmetic over ledgers (allocate cash in order, release in a fixed order), not if-then facts. The whole thing runs in one database transaction owned by one team, so an engine would add a DSL, debugging difficulty and a learning curve without removing any real complexity. I evaluated engine, saga and event sourcing in draft 3 and rejected them with reasons. The decision tree document says what would change my mind: non-engineers editing rules, dozens of order-dependent rules across teams, or cross-service operations.

Q. What is the "middle ground" you chose?

A declarative table with one row per command (roles, whether it reconciles, rule IDs), a shell that does the shared steps, and shared primitives for the hard logic. The table is a summary and a guard; domain logic stays in services. It is cheap, typed (the compiler forces a row per command) and easy to read.

Q. How do you correct a delivery without editing history?

Append-only rule WF-001. I add negative delivery rows that point at the delivery they reverse (reverses_id), reversing the most recent confirmations first. A database check enforces that a row is negative exactly when it reverses something. I also add a negative DELIVERY_REVERSAL charge against the original charge. Quantities are summed with sign, so existing reports and audit checks work unchanged.

B. The two features

Q. What happens to the money when a delivery is corrected?

The charge is reduced; anything paid above the reduced charge is released in the canonical order: cash, then credit, then points. Released cash becomes free cash, and the normal cash-priority rule re-applies it. That means when the missing units are confirmed later, the same cash pays for them with no manual step.

Q. How does moving a payment keep both accounts accurate?

The receipt is marked reversed through a link table, its allocations are cancelled by negative rows, and an identical replacement receipt (same amount, reference and original time) is created on the target customer. The action shell then reconciles both customers: A's charges reopen and may be re-settled from A's other free cash, B's cash settles B's charges, and reward points follow including clawback. The replacement keeps the original time so cash priority stays fair.

Q. Why keep the original timestamp on the replacement receipt?

Cash priority (ACC-003) orders by creation time. If the replacement took the move time, it would jump behind the target customer's other receipts and change which charges get paid first, which is not what happened in the real world.

C. Safety, testing and quality

Q. What is the audit review and why is it separate from the services?

After every action, about 25 invariant checks recompute things from raw rows (for example, allocations never exceed charges, no allocation crosses customers). If any fails, the transaction rolls back. It is deliberately independent code: it is an oracle that catches a service that is wrong. I added two checks: account-boundary (CUS-001) and per-reversed-receipt consistency.

Q. How do you handle concurrency?

Each action runs in one immediate SQLite transaction with optimistic version checks on the primary customer and idempotency by action id. A replayed request returns the original result. Limitation: for a move, the target customer's version is bumped but not checked.

Q. What are the limitations of your solution?

A receipt that funds a credit note cannot be moved (rejected with RECEIPT_FUNDS_CREDIT); the target customer's version is not checked on a move; correction only lowers quantities; ACC-004 / FUL-007 are my proposed rules that the business needs to confirm.

Q. How did you test it?

14 new tests drive the commands through HTTP. They cover the exact scenarios (10 to 6 then confirm 4; unpaid; part-paid; multiple deliveries reversed latest-first; returns limit; role and state rejections; move, move back, clawback, re-settlement from other cash, credit-funded receipt, idempotent replay) and a test that the rules table covers every command. The full suite is 265 tests and passes; after every action the tests assert all audit checks pass.

Q. What did you refactor and how did you avoid breaking things?

I made small steps with the full suite as a safety net: moved role checks into the table, moved reconcile into the shell, extracted releaseSettlement. Two real bugs appeared during this and were caught by tests: a bulk regex edit had removed my own authorize call, and then emptied the reconcile function, causing 8 failures. I found it by tracing and restored it. Lesson: do not bulk-edit with regexes, review every diff.

D. Process, AI and next steps

Q. How did you use AI in this task?

I used Claude Code as a pair programmer: to read and map the codebase, discuss design options, draft documents and write code and tests. I decided the design direction, reviewed what it produced and ran the checks. The "research" stage was a discussion with the agent about patterns, not external literature, and I said that in the evaluator notes.

Q. What would you do next?

Add automated UI tests for the two screens; support moving credit-funded receipts by reversing the credit first; check the target customer's version; confirm ACC-004 and FUL-007 with the business and fold them into the docs of record; and if more commands need cross-account or multi-step work, revisit the decision tree metrics.

Q. How would you decide when to invest in a bigger framework?

I use the decision tree: do non-engineers need to change rules without deploys; are there 50+ interacting rules owned by multiple teams; do operations span services; do we need replay. And metrics: decision nodes added per feature, repeated prologue code, audit failures that tests missed, time to add a pattern-following feature. If those trend badly, move up.

Quick files to know

Topic	File
Rules table	shared/src/command-rules.ts
Action shell and reconcile	server/src/actions/actions.service.ts
Move payment	server/src/accounts/accounts.service.ts (movePayment, releaseSettlement)
Correct delivery	server/src/fulfillment/fulfillment.service.ts (correctDelivery)
Audit checks	server/src/audit/audit.service.ts
Migration	server/drizzle/0006_productive_klaw.sql
Tests	server/test/corrections.test.ts
Docs	docs/architecture/04-final-architecture.md, decision-tree.md